

electron/prefetch-summaries.cjs · Code explanation (L222-233)

Kind: modified · Baseline: 46a8da0664d1 · Target: NovaDiff · Requested mode: exact · Effective mode: exact ·
Generated: 2026-05-19T23:16:58.455Z

What it does `startSummaryPrefetchWorker` initiates a single background pre-fetch loop for summary data. It first guarantees that any previously running worker is stopped (`stopSummaryPrefetchWorker`) and that stale cache entries are removed (`clearSummaryPrefetchCache`). A new `AbortController` is created, its reference stored in the module-level variable `prefetchAbort` , and its `signal` is passed to `runSummaryPrefetchLoop` . The function returns the promise from `runSummaryPrefetchLoop` , but attaches a `finally` handler that clears `prefetchAbort` once the loop finishes or is aborted, provided no newer controller has replaced it.

How it works 1. Cleanup – `stopSummaryPrefetchWorker()` halts any existing loop; `clearSummaryPrefetchCache()` wipes cached summaries. 2. **Abort setup*

- `const ac = new AbortController(); prefetchAbort = ac;` keeps a reference to the controller so other parts of the module can abort the loop. 3. **Execution*
- `runSummaryPrefetchLoop(opts, signal)` starts the asynchronous pre-fetch cycle, using the controller's `signal` to listen for abort requests. 4. **Finalization*
- The `finally` block runs after the loop resolves or rejects. It checks `prefetchAbort === ac` to avoid clearing the reference if a newer worker has already started, then sets `prefetchAbort` to `null` .

Notes for reviewers- The snippet assumes `stopSummaryPrefetchWorker` , `clearSummaryPrefetchCache` , and `runSummaryPrefetchLoop` are defined elsewhere; their side effects are not visible here. - `prefetchAbort` is a shared mutable state; ensure no race conditions arise if `startSummaryPrefetchWorker` is called concurrently. - The `finally` guard (`prefetchAbort === ac`) is crucial to prevent accidental nullification when a newer worker replaces the old one. - Unknown from the snippet: the exact behavior of `runSummaryPrefetchLoop` (e.g., how it uses the `signal`), and whether `stopSummaryPrefetchWorker` internally uses `prefetchAbort` .